SGit vaults in 90 seconds

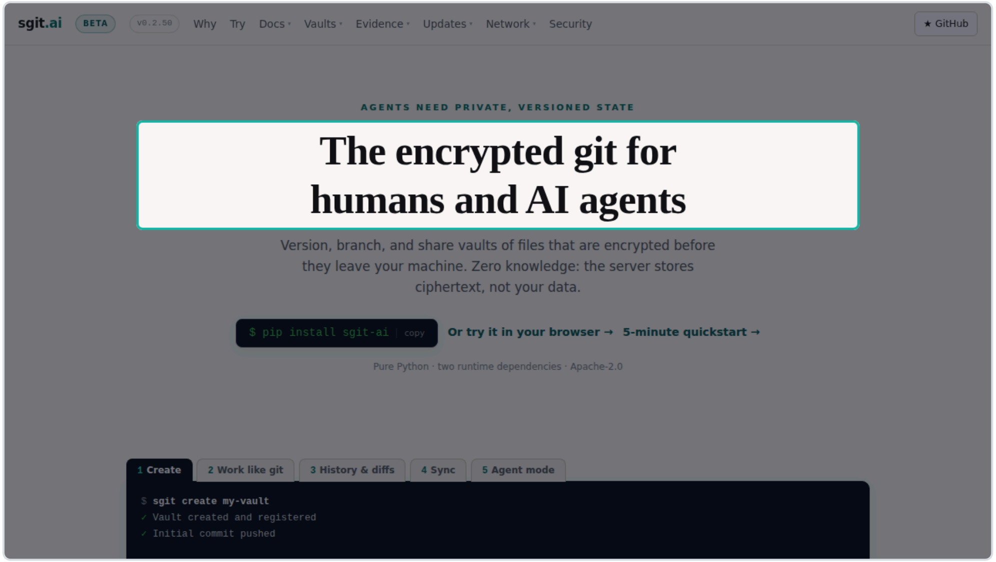
Encrypted, versioned folders the server can never read

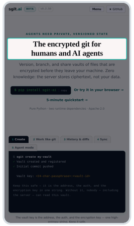
2026-09-01
made by a Claude Code agent, unattended
13 scenes · 01:57 landscape
source: reel.json

00:00 OPENING 7.1 S

What this video covers

SGit vaults, in ninety seconds. Encrypted, versioned folders the server can never read.





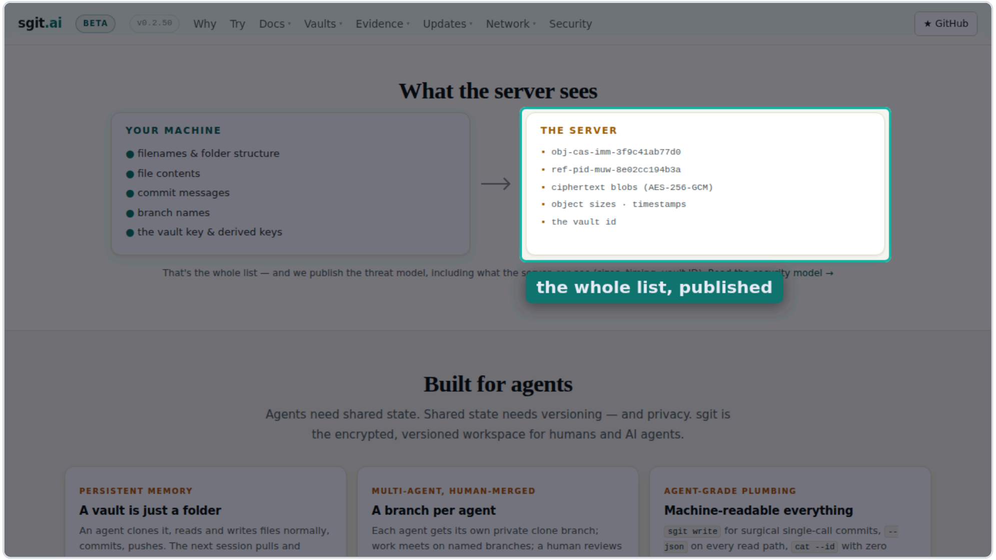
phone cut

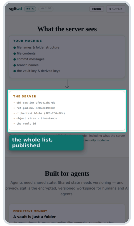
00:07 SCENE 1 OF 13 S01 7.4 S

Encrypted before it leaves

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

still · https://sgit.ai/ · spot heading:The encrypted git





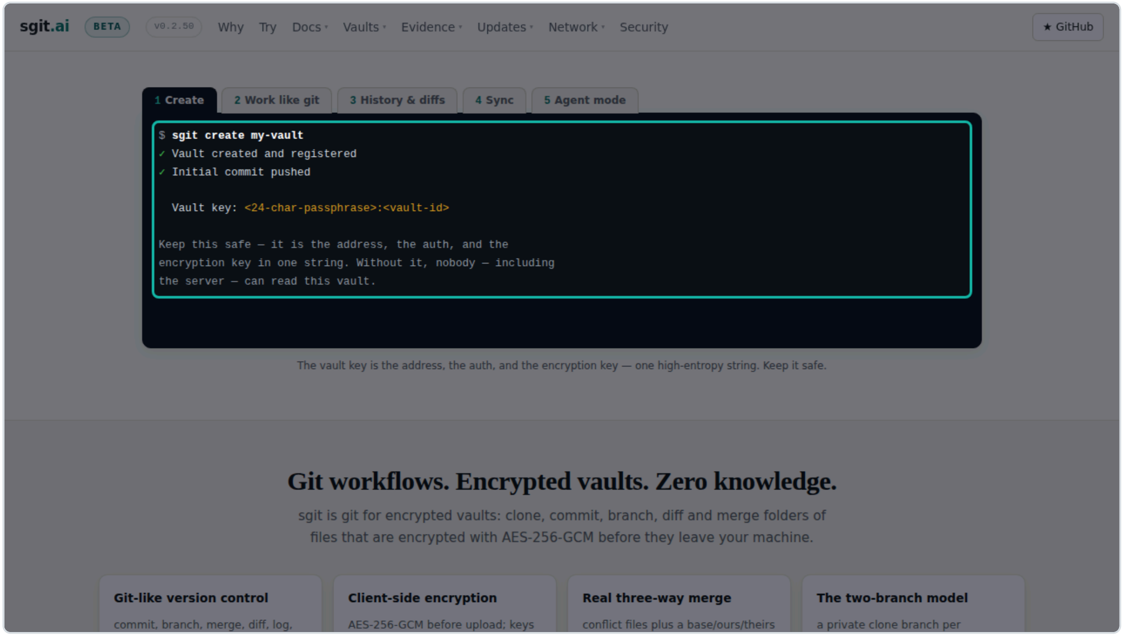
phone cut

`00:14` SCENE 2 OF 13 S02 8.8 S

The server sees ciphertext

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

`still · https://sgit.ai/ · scroll to “What the server sees” · spot right-column · label “the whole list, published”`

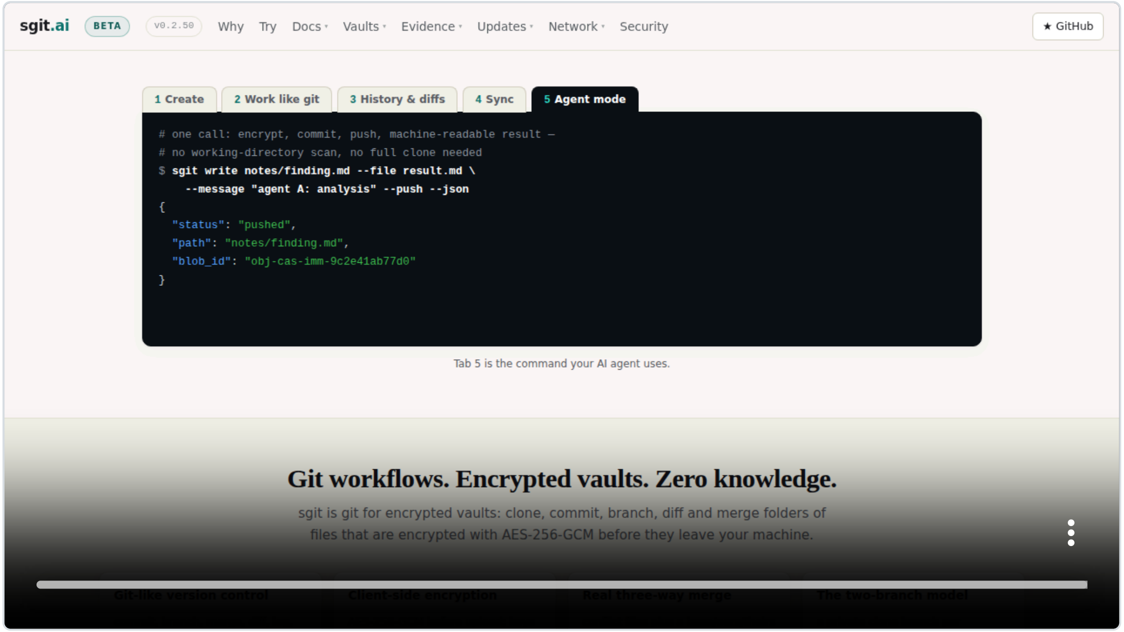


`00:23` SCENE 3 OF 13 S03 2.6 S

One command

Creating a vault is one command.

`still · https://sgit.ai/ · scroll to “Work like git” · spot terminal`

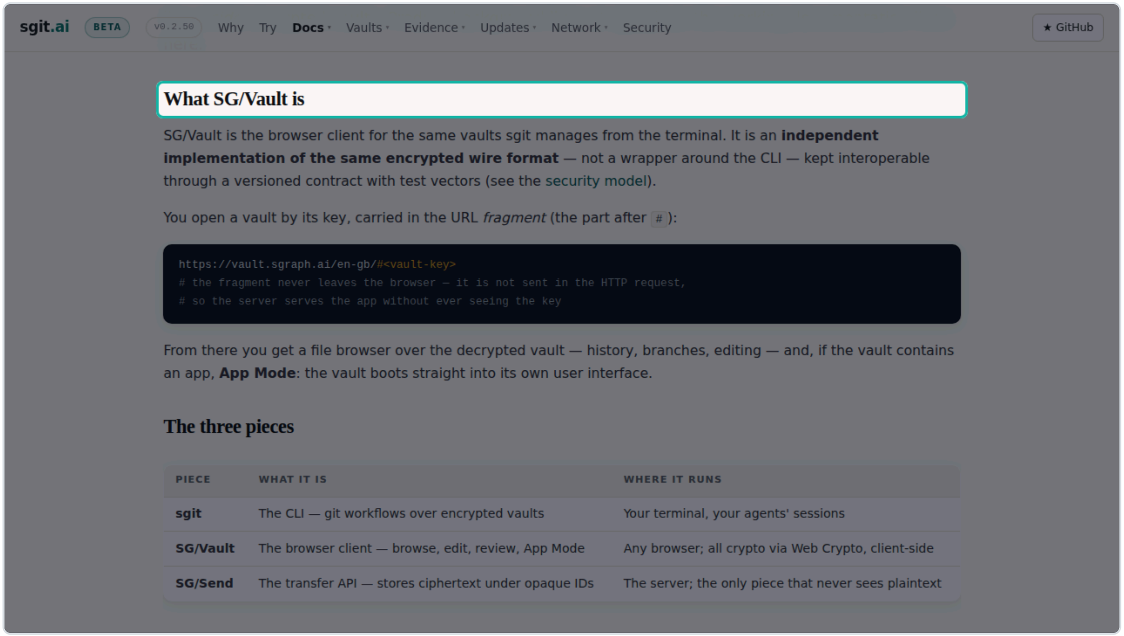


00:25 SCENE 4 OF 13 S04 7.3 S

Work like git

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

`clip · https://sgit.ai/ · scroll to “Work like git”`

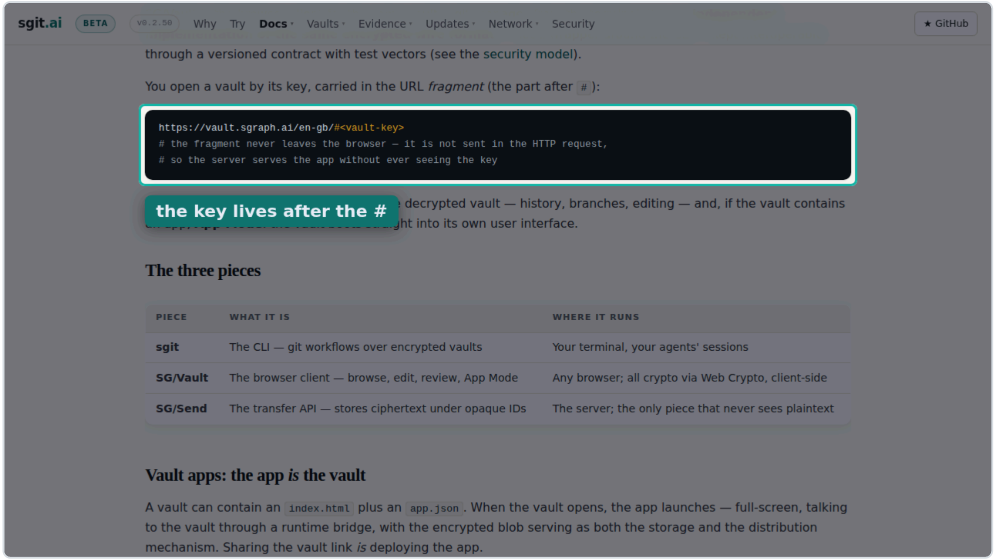


00:33 SCENE 5 OF 13 S05 8.5 S

Same vault, in the browser

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

`still · https://sgit.ai/vault/ · scroll to “What SG/Vault is” · spot heading:What SG/Vault is`



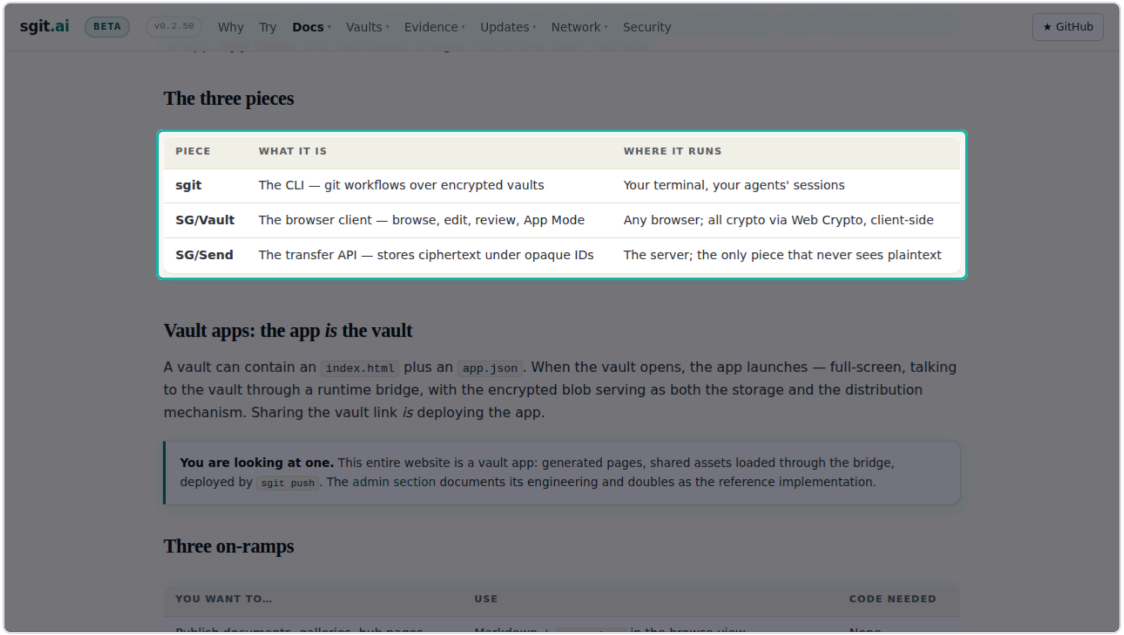
phone cut

00:41 SCENE 6 OF 13 S06 9.0 S

The key never leaves the browser

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

still · https://sgit.ai/vault/ · scroll to "You open a vault by its key" · spot code · label "the key lives after the #"

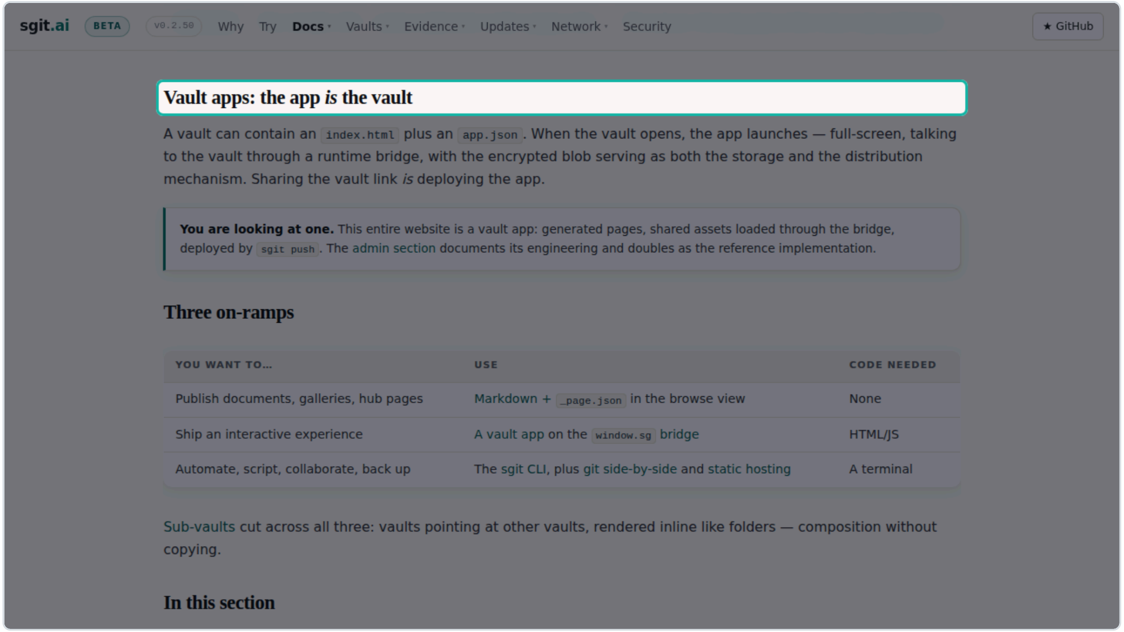


00:50 SCENE 7 OF 13 S07 12.3 S

Three pieces, one wire format

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

still · https://sgit.ai/vault/ · scroll to "The three pieces" · spot table

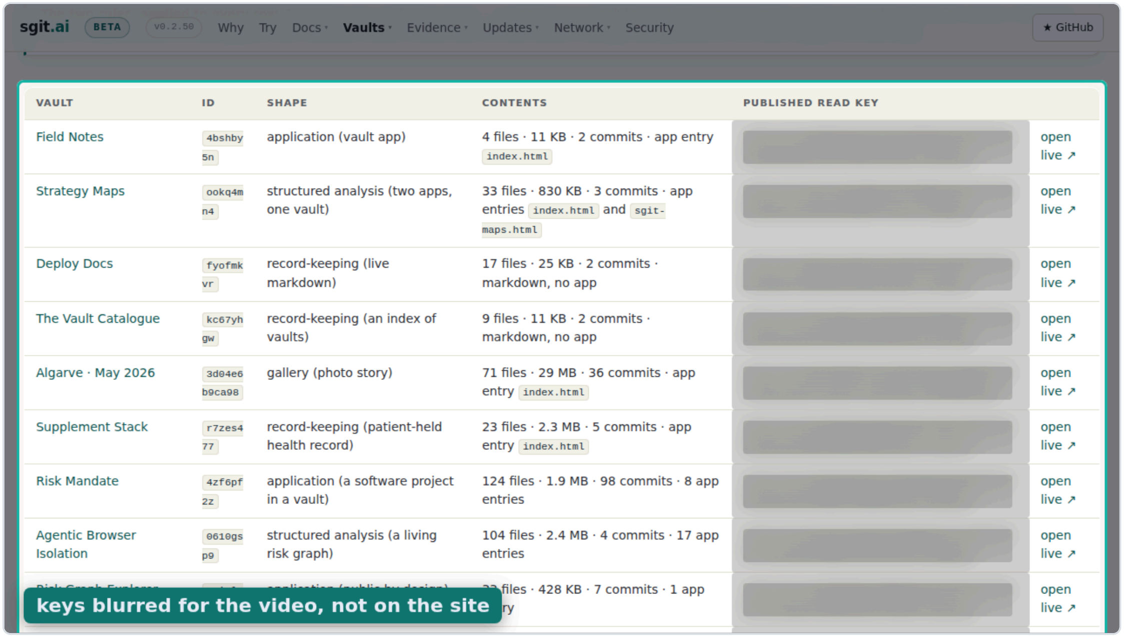


01:02 SCENE 8 OF 13 S08 5.5 S

The app is the vault

A vault can contain an app. Sharing the vault link is deploying the app.

`still · https://sgit.ai/vault/ · scroll to “Vault apps: the app is the vault” · spot heading:Vault apps: the app is the vault`

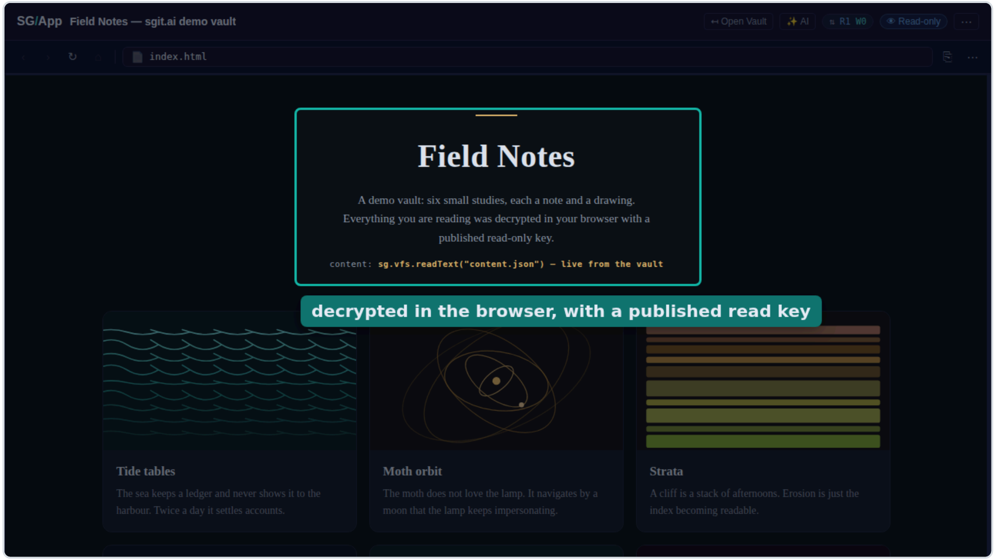


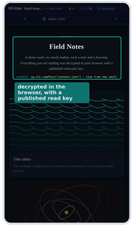
01:08 SCENE 9 OF 13 S09 8.4 S

Read keys, published on purpose

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

`still · https://sgit.ai/demos/vaults/index.html · scroll to “Published read key” · spot table · blur keys · label “keys blurred for the video, not on the site”`





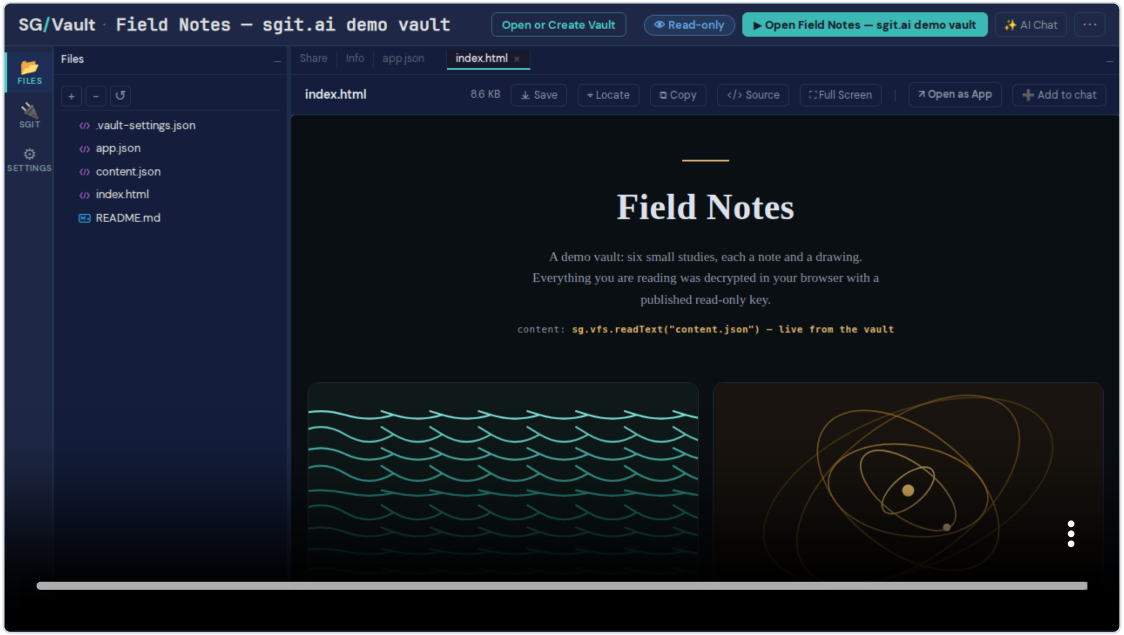
phone cut

01:16 SCENE 10 OF 13 S10 8.6 S

Decrypted in your browser

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

still · published vault “Field Notes” · spot rect · label “decrypted in the browser, with a published read key”

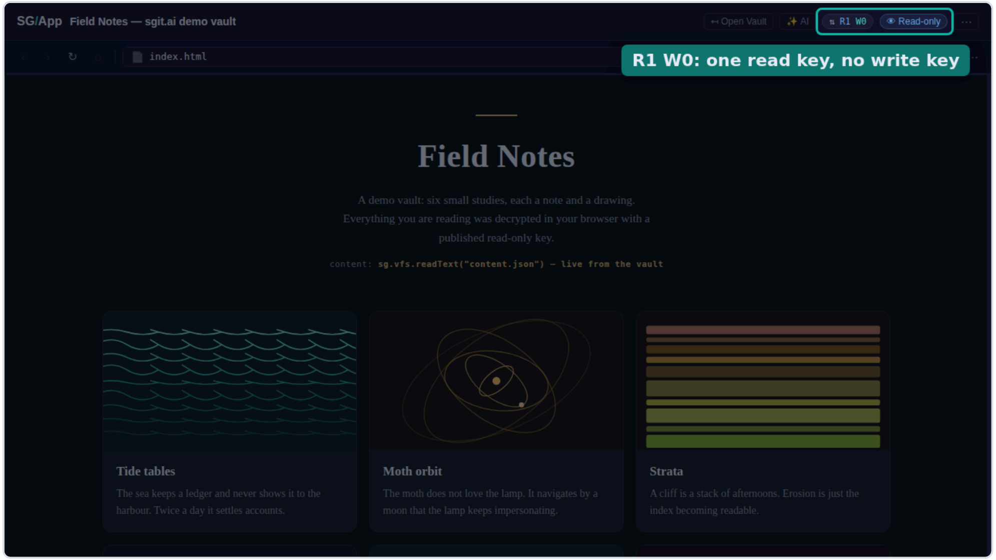


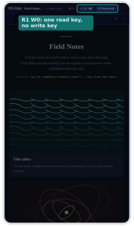
01:25 SCENE 11 OF 13 S11 8.6 S

Files, history, branches

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

clip · published vault “Field Notes”





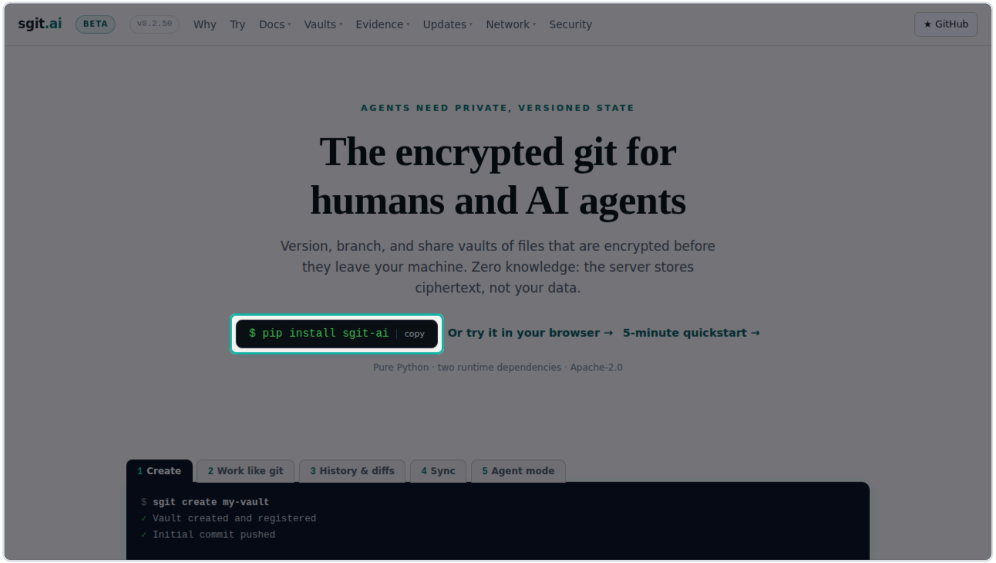
phone cut

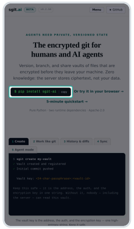
01:34 SCENE 12 OF 13 S12 5.3 S

Read-only by construction

And the badge says what the key allows. One read, zero writes.

`still · published vault “Field Notes” · spot badges · label “R1 W0: one read key, no write key”`





phone cut

01:39 SCENE 13 OF 13 S13 6.8 S

pip install sgit-ai

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

`still · https://sgit.ai/ · spot pip`

01:46 CLOSING 10.9 S

How this video was made

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

MADE BY a Claude Code agent, unattended

WHEN 2026-09-01

SCRIPT 13 scenes + title and closing · 257 words · reel.json written first

SCREENSHOTS 15 shot, 13 used · Playwright, headless · 93 s

NARRATION Kokoro-82M in the browser (2 workers) · 117.1 s of speech in 252 s · model load 14.7 s

RENDER video-creator, canvas + MediaRecorder, real time · 117 s

PIPELINE WALL CLOCK 479 s: capture 93 + compose and narrate 270 + render 117

API COST $0.00 · no LLM, TTS or image API calls

COMPUTE one 4-core container, ~10 min of CPU for both cuts · agent session tokens not metered here

Renders

CUT	LENGTH	SIZE	SLIDES	TTS	RECORD	DRIFT	PIPELINE
landscape 1280×720	01:57.16	10.8 MB	15	252 s	117.3 s	217 ms	387 s
shorts 1080×1920	00:46.01	5.4 MB	8	64 s	46.1 s	84 ms	118 s

Source of truth is `reel.json`. Stills in `images/` (desktop) and `images-shorts/` (phone); clips in `clips/`. Timecodes are the landscape cut's TTS durations.